

Перенос Рефала-5 на платформу Java, наброски

Коновалов А. В.

9–16 мая 2023 г.

Contents

Требования к реализации	2
Представление объектных выражений	2
Внутренний язык	4
Обоснование повышения автоматического местности	4
Про блоки	4
Льезонное представление	5
Льезонное поле зрения	6
Каноническое льезонное представление	8
Представление условий	9
Реализация	11
Реализация канонического льезонного представления	11
Области видимости Рефала-5 и Java	12
Компиляция функций	14
Компиляция образцов	15
Компиляция условий	17
Компиляция функций-выражений	18
Построение льезонной формы и повышение местности	19
Построение тривиальной льезонной формы	19
Вычисление форматов функций	20
Алгоритмы обобщённого сопоставления с образцом	22
Точное сопоставление формата функции с образцом $He : P$	22
Неточное сопоставление параметризованного выражения с образцом $E : P$	23
Сопоставление параметризованного выражения с форматом $E : He$	24
Алгоритм построения обобщения	25
Повышение местности и оптимизация условий	26
Генерация кода	27
Форматные и бесформатные функции	27
Компиляция метафункций	29
Прочее	29

Требования к реализации

- Совместимость с Рефалом-5 на уровне, достаточном для компиляции и выполнения программ SCP4, MSCP-A и crefal. Программы должны компилироваться и запускаться без дополнительной разметки (вроде форматов в Рефале Плюс).
- Массивное представление данных — это эксперимент, какую производительность оно покажет на программах, написанных для спискового представления.
- Неограниченная глубина рекурсии — реализация должна потреблять системный стек конечной глубины.

(Рефал-5 не удовлетворяет этому требованию, у него глубина стека для условий ограничена. Рефал-5λ тоже — при компиляции в Си++ использует системный стек для рекурсивного вызова рефал-машины в условиях. Рефал Плюс вроде тоже, но я не проверял.)

Представление объектных выражений

Промежуточное представление — массивное с дырками. Объектное выражение представляется типом `Slice`:

```
class Slice {
    Object at(int pos);
    int size();
    Slice slice(int left, int right);

    static Slice concat(Slice left, Slice right);
    static Slice concat(Slice slices...);
    static final Slice EMPTY;
    static Slice fromTerm(Object value);
}
```

По аналогии с Refal-Java, все значения, кроме `Slice` — символы:

Тип терма Рефала	Тип Java
Скобочный терм	<code>Slice</code>
Литера	<code>Character</code> , кодовая точка < 65 536
Литера	<code>r5jrt.BigChar</code> , кодовая точка ≥ 65 536
Макроцифра	<code>Long</code>
Символ-слово	<code>String</code>

Произвольный класс в объектном выражении допустим, функция `Type` для него будет возвращать `00`.

Значение `null` в срезе недопустимо!

Интерфейс класса `Slice` описан минималистично. В реализации могут появиться новые методы из соображений оптимизации.

Также, возможно, срез будет реализовывать интерфейс некоторых стандартных контейнеров.

- `Object at(int pos)` возвращает терм в данной позиции. Допустимые индексы от 0 до `size()`.
- `int size()` возвращает количество элементов в среде.
- `Slice slice(int left, int right)` вырезает новый срез из текущего, `left` и `right` описывают диапазон *включительно*, длина нового среза `right - left + 1`.
- `static Slice concat(Slice left, Slice right)` конкатенирует два среза.
- `static Slice concat(Slice... slices)` конкатенирует несколько срезов.
- Константа `Slice EMPTY` и фабрика `Slice fromTerm(Object value)` создают новый срез, `value != null`.

Предполагается такая реализация:

```
class Slice {
    Object[] items;
    int left, right;
}
```

- Массив `items` в общем случае состоит из непрерывной последовательности значений `null` (0 и более раз), непрерывной последовательности ненулевых значений (1 и более) и ещё одной непрерывной последовательности нулей (0 и более раз). Нули слева и справа образуют дырки.
- Индексы `left` и `right` выбирают диапазон *включительно*.

Конкатенация `concat(left, right)` выполняется эффективно, если у одного из аргументов с краю находится дырка достаточной длины, чтобы вместить другой аргумент.

Иначе, создаётся новый срез длиной $2 * (\text{left.size()} + \text{right.size()})$ с левой дыркой длиной `right.size()` и правой длиной `left.size()`.

Обоснование: такая стратегия обеспечит амортизированное линейное время в типичных случаях вида

```
F {
    ... = ... <F...>;
}
```

```
G {
    (e.Асс) ... = <G (e.Асс ...) ...>;
}
```

Конкатенация нескольких значений `concat(...slices)` выполняет два прохода по массиву аргументов: на первом подсчитывает суммарный размер (и удаляет пустые срезы), на втором — ищет срез с дырками достаточного размера, чтобы вместить все предшествующие или последующие куски. Если подобный срез не найден, создаётся срез вместительностью вдвое большей, чем его длина.

Длины дырок вычисляются следующим образом. Пусть длины аргументов равны $\ell_1, \ell_2, \dots, \ell_n$. Если n чётное то правая дырка имеет длину $\ell_1 + \dots + \ell_{n/2}$, левая, соответственно, $\ell_{n/2+1} + \dots + \ell_n$. Если нечётное — правая дырка — $\ell_1 + \dots + \ell_{\lfloor n/2 \rfloor} + \lceil \ell_{\lfloor n/2 \rfloor} / 2 \rceil$, левая дырка — $\lfloor \ell_{\lfloor n/2 \rfloor} / 2 \rfloor + \ell_{\lfloor n/2 \rfloor + 1} + \dots + \ell_n$.

Эти формулы являются обобщением правила вычисления дырок для `concat(left, right)`.

Метод `fromTerm(o)` можно оптимизировать. Вместо создания массивов единичной длины для одиночных термов можно в статической переменной хранить массив, в него добавлять новый терм и создавать срезы единичной длины, ссылающийся на этот массив.

Внутренний язык

Обоснование повышения автоматического местности

Рефал-5 непосредственно не подходит для массивного представления данных — типичные форматы функций имеют вид `<F t (e) s e t> == (e s) e`, т.е. предполагают, что один из `e`-аргументов при передаче в функцию (один из `e`-результатов при возврате) можно не заворачивать в скобки.

Примером являются, в частности, встроенные функции `Type`, `Lenw`, `First`, `Last`, `Open` и т.д. Для массивного представления данных эта идиома приведёт к потерям производительности вплоть до повышения порядка алгоритмической сложности.

Пример:

```
Fab {
  (e.Acc) A e.Rest = <Fab (e.Acc B) e.Rest>;
  (e.Acc) s.X e.Rest = <Fab (e.Acc s.X) e.Rest>;
  (e.Acc) /* пусто */ = e.Acc;
}
```

Сложность на списковом представлении $O(n)$, сложность на массивном представлении будет $O(n^2)$ и дырки тут помочь не в силах.

Избежать таких форматов нельзя (т.е. потребовать от пользователей заворачивать все `e`-аргументы и `e`-результаты в скобки), т.к. согласно первому требованию уже написанные программы на Рефале-5 должны работать без изменений. И работать они должны, что подразумевается, эффективно.

Добавлять ручную (как в Рефале Плюс) форматы тоже нельзя по тому же первому требованию.

Поэтому выход — форматы у функций должны быть, но во входном языке их быть не должно. Следовательно, Рефал-5 должен переводиться во внутренний язык с автоматическим повышением местности и коместности.

Про блоки

Сразу упомянем блоки, чтобы потом забыть о них. Блок является синтаксическим сахаром, который тривиально преобразуется в вызов вспомогательной функции, что, собственно, и будет делать компилятор. В массивном представлении это преобразование делается без какой-либо потери эффективности.

Пример:

```

F {
  (e.X) (e.X), A : { A = e.X e.X }
}

```

Наивное рессахаривание в Рефале-5 даст программу

```

F {
  (e.X) (e.X) = <F1 (e.X) A>;
}

```

```

F1 {
  (e.X) A = e.X e.X;
}

```

которая будет выполнять лишнее копирование значения `e.X` в `F1`. Попытка передать `e.X` дважды

```

F {
  (e.X) (e.X) = <F1 (e.X) (e.X) A>;
}

```

```

F1 {
  (e.X) (e.X) A = e.X e.X;
}

```

приведёт к лишнему сопоставлению с повторной переменной. Если же блоки реализовывать нативно, то исходный вариант будет выполняться без лишних копирований и сопоставлений.

(В Рефале-5λ и в рессахаривателе из `refal-5-framework` реализован наивный вариант, который может приводить к потерям эффективности в подобных редких случаях.)

В массивном представлении наивный вариант рессахаривания будет эффективен.

Льезонное представление

Когда у функций одно возвращаемое значение, при записи выражения можно в ту точку, это это значение требуется, записать вызов функции. Если же у функции возвращаемых значений несколько (а такого требует массивное представление), то эти значения в выражении могут встречаться в разных местах, в том числе и не по соседству. Нужен другой формализм для записи выражений.

Проще всего выписать вызов функции отдельно, поименовать его возвращаемые значения и в выражении на местах использования этих значений написать их имена (имена переменных).

Т.к. эта процедура будет выполняться с каждым вызовом функции в выражении, то мы придём к перечню вызовов с поименованными возвращаемыми значениями и пассивным выражением без вызовов функций, в котором используются имена возвращаемых значений.

Введём обозначения для этого формализма.

Далее в квадратных скобках `[ e1 , e2 , ..., eN ]` мы будем записывать кортеж выражений или имён переменных.

Функция может принимать несколько аргументов и возвращать несколько значений, поэтому кортеж будет использоваться и при записи левой части функции (кортеж образцов), и при записи

вызовов функций из правой части (кортежи их аргументов и имён возвращаемых значений), и при записи результата самой функции (кортеж возвращаемых значений в правой части).

Пока мы ограничимся рассмотрением базисного Рефала (расширенного новым формализмом), об условиях мы поговорим позже.

Вызов функции в новом формализме будем записывать как

$$\langle F [E_1, E_2, \dots, E_m] \rangle :: [v_1, v_2, \dots, v_n]$$

т.е. функция принимает m аргументов и возвращает n значений. Т.к. функция принимает кортеж всегда, то удаление квадратных скобок вокруг аргументов путаницу не внесёт:

$$\langle F E_1, E_2, \dots, E_m \rangle :: [v_1, v_2, \dots, v_n]$$

Предложение в этом формализме будем записывать как

$$\begin{aligned} &[m_1 : P_1, m_2 : P_2, \dots] \\ &, \langle F_1 E_{11}, E_{12}, \dots \rangle :: [v_{11}, v_{12}, \dots] \\ &, \langle F_2 E_{21}, E_{22}, \dots \rangle :: [v_{21}, v_{22}, \dots] \\ &\dots \\ &= ER_1 : m_1', ER_2 : m_2', \dots; \end{aligned}$$

Здесь m_i означает тип соответствующего аргумента: e , t или s .

Квадратные скобки здесь использовать желательно, т.к. запятая разделяет не только компоненты кортежа, но и вызовы функций. Выражения $E_{i,j}$ и E_j^R — пассивные, т.е. содержат только символы, скобки и имена переменных.

Переменные $v_{i,j}$, используемые как имена возвращаемых значений вызовов F_i , не могут быть повторными: они не могут совпадать с именами переменных из образцов P_m , не могут совпадать с переменными $v_{k,l}$, где $k < i \vee (k = i \wedge l < j)$.

Вслед за Турчиным, переменные $v_{i,j}$ мы будем называть *переменными связи* или *льезонными переменными* (*liaison variables*)¹, а запись предложений в этой форме — *льезонным представлением*.

Аргументы и результаты функций мы будем подразделять на те же три типа, что и у переменных: s — любой символ, t — любой терм и e — любое выражение, поэтому при их записи мы будем использовать обычную нотацию для переменных Рефала-5: $\langle \text{тип} \rangle . \langle \text{индекс} \rangle$.

Льезонное поле зрения

Семантика базисного Рефала определяется через поле зрения — выражение с вызовами функций, в котором на каждом шаге выбирается ведущий вызов и заменяется на новое выражение (возможно, с вызовами функций). В списковой реализации эта семантика воплощается непосредственно и имеет реализацию, не требующую рекурсии.

Третье требование к реализации — неограниченная глубина рекурсии в программах на Рефале, не требующая рекурсию в реализации. Можно ли описать семантику поля зрения для массивной реализации объектных выражений и льезонного представления программы? Притом с эффективной реализацией?

Поступим по аналогии. Классическое поле зрения представляет собой *активное* выражение — выражение Рефала, построенное из символов, структурных скобок и скобок конкретизации.

¹V. F. Turchin. The School «Metacomputation in the Language Refal». Obninsk, July 11-23, 1990.

Правая часть предложения — *результатное* выражение — отличается от активного лишь наличием переменных, оно задаёт множество активных выражений. Конкретный экземпляр множества определяется значениями переменных, полученными при сопоставлении с образцом.

Результатное выражение нужно, чтобы получить активное выражение, на которое заменится вызов функции в поле зрения. Активное выражение, находящееся в поле зрения, никакое значение не вычисляет — его окончательное значение, когда оно станет пассивным, просто отбрасывается, поле зрения вычисляется лишь ради побочного эффекта.

Стало быть, чтобы получить лезонное поле зрения, нам нужно (а) описать структуру данных поля зрения и (б) описать операцию замены вызова функции на его результат.

Содержимое поле зрения отличается от результатной части предложения отсутствием переменных — убираем из лезонной части все *свободные* переменные — те, которые приходят из образца, оставляем лишь *связанные* переменные — лезонные переменные или переменные связи.

Окончательное значение вычислений не нужно — функции вычисляются ради побочного эффекта — убираем часть = ER1, ER2,

Получаем поле зрения в следующей форме:

```
<F E11, E12, ... > :: [v.11, v.12, ... ],
<G E21, E22, ... > :: [v.21, v.22, ... ],
...
<Z En1, En2, ... > :: [v.n1, v.n2, ... ]
```

В выражениях $E_{i,j}$ могут использоваться только переменные $v_{k,l}$, где $k < i$. В выражениях $E_{1,j}$ никакие переменные не могут встречаться вообще, аргументы первого вызова функции — объектные выражения.

Фактически, поле зрения можно рассматривать как стек.

Теперь опишем шаг вычислений. Активный вызов определяется просто — это вызов на верхушке стека. Пусть у функции F на верхушке сработало предложение

```
[m1 : P1, m2 : P2, ... ]
, <F1 E11', E12', ...> :: [w.11, w.12, ... ]
, <F2 E21', E22', ...> :: [w.21, w.22, ... ]
...
, <Fk Ek1', Ek2', ...> :: [w.k1, w.k2, ... ]
= ER1' : m1', ER2' : m2', ... ;
```

Тогда мы должны:

- удалить со стека вызов <F E11, E12, ... > ,
- поместить на стек вызовы $F_1, \dots, F_k$ (на новой верхушке F_1),
- в оставшейся части стека заменить переменные $v_{1,1}, v_{1,2}, \dots$ на выражения $E_1^{R'}, E_2^{R'}, \dots$ соответственно.

Поле зрения примет вид

```
<F1 E11', E12', ...> :: [w.11, w.12, ... ],
<F2 E21', E22', ...> :: [w.21, w.22, ... ],
...
```

```

<Fk Ek1', Ek2', ...> :: [w.k1, w.k2, ... ],
<G E21", E22", ... > :: [v.21, v.22, ... ],
...
<Z En1", En2", ... > :: [v.n1, v.n2, ... ]

```

Двойной штрих здесь означает подстановку в $E_{i,j}$ выражений $E_j^{R'}$.

Промежуточный вывод. Получилось описать семантику лезонного поля зрения. И, похоже, её можно реализовать без рекурсии. Сложность вызывает лишь реализация представления выражений с лезонными переменными и эффективной операцией подстановки.

Каноническое лезонное представление

Вызов функции в лезонном представлении в качестве аргументов принимает выражения, содержащие лезонные переменные:

```

<F E1, E2, ..., Em> :: [v.1, v.2, ..., v.n]

```

Выражения E_i должны допускать эффективную операцию подстановки значения вместо лезонной переменной, кроме того, после подстановки значений вместо всех переменных, мы должны получать объектное выражение. Т.е. мы должны иметь какую-то структуру данных, которая способна обратиться в `Slice`, рассмотренный в начале документа.

Оттолкнёмся от простого: какой самый простой случай замены, дающей объектное выражение? Возможно два случая:

- Само выражение является объектным, лезонных переменных не содержит, заменять в нём нечего.
- Выражение является переменной, подстановка заменяет его на объектное выражение (т.е. выражение без переменных).

Здесь же можно упомянуть и третий простой случай:

- Выражение является переменной, подстановка заменяет его на другую переменную. Да, он не даёт объектного выражения, но он простой.

Хорошая новость в том, что сложный случай (выражение E_i не является ни объектным, ни одиночной переменной) можно выразить через простой: само выражение с переменными можно рассматривать как функцию, параметрами которой являются переменные, а результатом — результат подстановки переменных в выражение.

Трактуя параметризованные выражения как функции, можно получить лезонное представление, в котором аргументами функций будут либо объектные выражения, либо лезонные переменные. Вызов функции вида

```

<F E1, E2, ..., Em> :: [v.1, v.2, ..., v.n]

```

будет представлен как

```

<{E1} w.11, w.12, ... > :: u.1,
<{E2} w.21, w.22, ... > :: u.2,
...
<{Em} w.m1, w.m2, ... > :: u.m,
<F u.1, u.2, ..., u.m > :: [v.1, v.2, ..., v.n]

```

Например,

```
<F (s.2 A e.1) e.3, e.1 e.3, ((s.2 e.1))> :: [e.4, t.5, s.6]
```

можно представить в виде

```
<{(s.#1 A e.#2) e.#3} s.2, e.1, e.3> :: e.7,  
<{e.#1 e.#2} e.1 e.3> :: e.8,  
<{((s.#1 e.#2))} s.2, e.1> :: t.9,  
<F e.7, e.8, t.9> :: [e.4, t.5, s.6]
```

Здесь индексы $v.\#n$ показывают номер параметра, который подставляется в это место выражения.

Аналогично трансформируются и правые части вида $= E_1^R, E_2^R, \dots$.

Такое представление и правых частей, и поля зрения (назовём его **каноническим**) позволяет эффективно выполнять замену вызова функции на верхушке стека поля зрения на её результат.

Можно предположить, что функции вида $\{expr\}$ компилятор генерирует каким-то особым образом: принимает несколько объектных выражений, и вызывая методы `Slice`, формирует результат.

Для некоторых распространённых выражений (например, $\{e.\#1 \ e.\#2\}$, означающей простую конкатенацию) в рантайме могут быть заранее заготовлены функции, чтобы их каждый раз не генерировать заново. Представляет интерес, какие именно функции на практике окажутся распространёнными и достойными включения в рантайм.

Представление условий

Вписать условия в псевдокод несложно:

```
F {  
  ...  
  
  [m1 : P1, m2 : P2, ... ]  
    , <G1 u.11, u.12, ... > :: [v.11, v.12, ... ]  
    ...  
    , <Gk u.k1, u.k2, ... > :: [v.k1, v.k2, ... ]  
    ? [w.1, w.2, ... ] :: [Q1, Q2, ... ]  
    , <Gk' u.k'1, u.k'2, ... > :: [v.k'1, v.k'2, ... ]  
    ...  
    = u.r1, u.r2, ... ;  
  
  ...  
}
```

Новым здесь будет только то, что сопоставляется не выражение с образцом, а кортеж переменных w_i с кортежем образцов Q_i . Переменные обусловлены канонической формой, а кортеж — очевидная оптимизация, когда и выражение слева, и образец справа имеют общий формат (нетривиальное обобщение).

Интересным здесь является то, как будет выполняться это условие.

Пусть лидирующим в поле зрения является вызов этой функции F , окажется применимым предложение с условием и (для простоты) условие в предложении только одно:

```
<F Oe1, Oe2, ... > :: [x.1, x.2, ... ],
...
{стек}
```

Здесь Oe_i — объектные выражения. Вызов функции F будет заменён на

```
<G1 u.11, u.12, ... > :: [v.11, v.12, ... ],
...
<Gk u.k1, u.k2, ... > :: [v.k1, v.k2, ... ],
<F' {состояние}, w.1, w.2, ... > :: [x.1, x.2, ... ],
...
{стек}
```

Здесь {состояние} — состояние вызова F в данном предложении: аргументы функции, номер текущего предложения, номер текущего условия и сопоставленные переменные образца (включая открытые переменные).

Пусть вычисление дошло до функции F' :

```
<F' {состояние}, Oe'1, Oe'2, ... > :: [x.1, x.2, ... ],
...
{стек}
```

Функция F' осуществляет сопоставление с образцом условия, и, в зависимости от результата, может перейти в одно из нескольких состояний.

Если сопоставление с образцом успешно, то подставляется остаток предложения после условия:

```
, <Gk' u.k'1, u.k'2, ... > :: [v.k'1, v.k'2, ... ]
...
{стек | x.1 → u.r1, x.2 → u.r2, ... }
```

и в оставшейся части стека делаются замены переменных x_i на переменные u_i^R из = u.r1, u.r2,

Если сопоставление с образцом неуспешно, то возобновляется выполнение функции F :

```
<F Oe1, Oe2, ... > :: [x.1, x.2, ... ],
...
{стек}
```

однако, с учётом {состояния} — если в образце предложения были открытые переменные, то делается попытка другого сопоставления, если не было — управление передаётся на следующее предложение.

Случай с несколькими условиями в предложении рассматривается аналогично.

Реализация

Реализация канонического лезонного представления

Вспомним, что объектное выражение представляется как `Slice`, который реализует абстракцию массива: метод `at(pos)` отображает отрезок целых чисел от 0 до `size()` - 1 на значения типа `Object`.

А сами объектные термы и есть `Object`'ы: скобочный терм представлен как экземпляр `Slice`, символ — как любой другой тип.

В реализации удобно представлять параметры функций как массивы типа `Object[]`: s-параметры при этом никогда не будут хранить объекты `Slice` (это упростит сопоставление с образцом), t-параметры могут иметь любое значение, e-параметры — только значение типа `Slice` (да, потребуется приведение типа).

Поскольку и аргументы, и возвращаемые значения — переменные, возвращаемое значение можно представлять как ссылку на элемент массива аргументов какой-либо функции. В Java, в отличие от C++, ссылки на элементы массива непосредственно не представимы, поэтому потребуется хранить ссылку на сам массив + индекс элемента, который надо обновить.

В общем случае, лезонная переменная в поле зрения может встречаться ноль или более раз. При простой компиляции из базисного Рефала такое невозможно, неиспользуемые или размножившиеся переменные могут появиться либо при оптимизации, либо при компиляции условий с оптимизацией (см. далее).

Общий случай возвращаемого значения предполагает использование списков пар массив + индекс. Список может быть как пустым, так и содержать несколько звеньев. В типичном случае звено в нём будет одно.

Возвращаемое значение можно описать так:

```
class ReturnValue {
    public Object[] args;
    public int argOffset;
    public ReturnValue nextSlot;

    static void assign(ReturnValue ret, Object value) {
        while (ret != null) {
            ret.args[ret.argOffset] = value;
            ret = ret.next;
        }
    }

    static ReturnValue join(ReturnValue x, ReturnValue y) {
        if (x != null) {
            if (y != null) {
                ReturnValue last = x;
                while (last.next != null) {
                    last = last.next;
                }
                last.next = y;
            }
        }
    }
}
```

```

        return x;
    } else {
        return y;
    }
}
}

```

Метод `assign(ret, value)` присваивает вычисленное значение. Он вынужден быть статическим, т.к. возвращаемое значение может быть `null`, если оно не используется (так может получиться в результате оптимизации условий).

Метод `join(x, y)` объединяет два возвращаемых значения в одно. Он потребуется в случае

```
= u.rl, ... , u.rj, ... , u.rj, ...
```

Функции, при компиляции в Java, разумно представлять как классы, а их вызовы в поле зрения — как экземпляры этих классов. Базовый класс для функции, и, соответственно, её вызова в поле зрения может иметь вид:

```

abstract class Function {
    public Object[] args;
    public ReturnValue[] returns;
    protected int go_to; /* см. далее */
    Function next; /* ссылка на следующий узел в поле зрения */

    public abstract void eval(r5jrt.Runtime runtime);
}

```

Открытые поля нас смущать не должны — эти классы пишутся не для человека, а для компилятора.

Аргумент `runtime` метода `eval()` — это экземпляр текущего состояния рефал-машины, включающий поле зрения и копилку.

Области видимости Рефала-5 и Java

Область видимости переменных — предложение, с ними всё понятно.

В Рефале есть две области видимости для функций:

- глобальная область видимости для функций, помеченных `$ENTRY` (будем называть их *entry-функциями*),
- и область видимости файла для функций без этой пометки (будем называть их *локальными*).

В разных файлах могут находиться одноимённые локальные функции и это не приводит к конфликту имён. Несколько одноимённых `entry-функций` в одной программе находиться, понятное дело, не могут.

А ещё есть синтаксическое требование, что в каждом файле должна быть как минимум одна `entry-функция`, иначе такой файл просто бесполезен (ни одну из функций, определённых в нём, вызвать невозможно).

В Java область видимости классов иерархическая: пакеты, классы (для вложенных классов), методы (для локальных классов). На каждом уровне иерархии (кроме локальных классов) можно определить доступ к сущности: пометить ключевым словом `public` видимое глобально, никак не пометить или пометить чем-то иным, чтобы глобального доступа не было (детали нас не интересуют).

Каждый уровень иерархии, кроме глобальной области видимости, поименован: имена есть у пакетов и классов.

Задача состоит в том, чтобы отобразить области видимости Рефала на области видимости Java. Причём, задача ограничения доступа не стоит — достаточно предотвращать конфликты имён для локальных функций из разных файлов.

Напрашивается решение сопоставить entry-функциям классы в глобальной области видимости, либо в пакете с фиксированным именем. Мы выберем второй вариант: классы функций будем помещать в пакет `r5j`.

Остаётся вопрос с локальными функциями. В реализации Рефала-5 с ними проблем нет — каждый файл исходного текста (расширение `.ref`) компилируется в файл байткода (расширение `.rsl`), путь каждого файла уникален в файловой системе.

Можно в имена классов для локальных функций добавлять путь исходного файла, соответствующим образом задекорированное (символы, недопустимые в именах, заменяются на некоторые escape-последовательности). Но это коряво и безобразно. Есть эстетически лучшее решение.

Вспомним, что в каждом исходном файле есть как минимум одна entry-функция, а имена entry-функций в программе должны быть уникальными. К именам локальных функций можно добавлять имя одной из entry-функций файла (например, самой первой), причём выразить это средствами Java!

Мы предлагаем добавлять в класс entry-функции (для определённости это будет самая первая в файле) локальные функции как статические классы.

Вот так. Для исходного файла на Рефале

```
F1 { ... }
$ENTRY F2 { ... }
F3 { ... }
$ENTRY F4 { ... }
F5 { ... }
```

будет создан файл `r5j/F2.java`, соответствующий первой entry-функции. В нём также будут находиться реализации всех локальных функций файла:

```
package r5j;

public class F2_ extends r5jrt.Function {
    ...

    public static class F1_ extends r5jrt.Function { ... }
    public static class F3_ extends r5jrt.Function { ... }
    public static class F5_ extends r5jrt.Function { ... }
}
```

Во втором файле `r5j/F4.java` будет только реализация второй entry-функции:

```
package r5j;
```

```
public class F4_ extends r5jrt.Function { ... }
```

Имя функции на Рефале может быть произвольным словом, однако в Java есть ключевые слова. Исходник может иметь, например, такой вид:

```
$ENTRY class { ... }  
$ENTRY if { ... }
```

и его тоже надо уметь компилировать.

Кроме того, имена функций могут содержать знак `-`, который в именах классов Java недопустим. Так что, имена надо декорировать.

Предлагается следующая схема, использующая `_` в качестве escape-символа:

- знак `-` заменяется на `_d`,
- знак `_` заменяется на `_u`,
- в конец имени добавляется знак `_`.

Библиотечные классы, которые предполагается использовать в сгенерированном коде (например, `Object`, `String`), на прочерк не заканчиваются — это предотвращает конфликт имён с ними.

Дополнительно мы избегаем конфликта имён, используя пакет `r5j`.

Примеры преобразования имён:

идентификатор Рефала	идентификатор Java
A-B_C	r5j.A_dB_uC_
x-----	r5j.x_d_d_d_d_d_d_d_
class	r5j.class_
Object	r5j.Object_

Компиляция функций

Функция компилируется по следующему шаблону:

```
class <Имя> extends r5jrt.Function {  
    public <Имя>() {  
        args = new Object[<число>];  
        returns = new ReturnValue[<число>];  
        go_to = 0  
    }  
  
    int[] e = new int[<число элементов> + <число е-переменных>];  
    Slice[] s = new Slice[<число е-аргументов> + <число скобочных термов>];  
  
    @Override  
    public void eval(r5jrt.Runtime runtime) {
```

```

while (true) {
    switch (go_to) {
        case 0:
            < базовый блок >
        case 1:
            < базовый блок >
        ...
    }
}
}
}
}

```

С конструктором всё очевидно.

Поля e и s хранят информацию о сопоставленных элементах в образце, в том числе и о значениях переменных. Если функция не имеет условий, то эти поля можно сделать локальными переменными метода `eval()`.

Каждый сопоставленный элемент описывается либо кортежем $\langle e_i, s_j \rangle$ — для символов, скобочных термов, s - и t -переменных, либо кортежем $\langle [e_{i'}, e_{i''}], s_j \rangle$ — для e -переменных. Здесь s_j — это срез, в котором находится элемент, e_i — позиция терма в срезе, $e_{i'}, e_{i''}$ — позиции первого и последнего термов e -переменной. При этом, если $e_{i'} > e_{i''}$ — переменная является пустой ($e_{i'} = e_{i''}$ — переменная длины 1).

Разные индексы i и j в кортежах подчёркивают, что один и тот же срез s_j может использоваться с разными элементами e_i .

Неотождествлённые части образцов представляются в программе как открытые интервалы $\langle (e_l, e_r), s_j \rangle$, где e_l и e_r — отождествлённые элементы перед и после «дырки», в их роли могут выступать и концы e -переменных.

Компиляция образцов

Подробно рассматривать сопоставления с элементами мы не будем, кроме открытых переменных.

Бесконечный цикл `while (true)` и переключатель по полю `go_to` фактически используются для моделирования безусловного перехода, которого в Java нет. (Но есть зарезервированное ключевое слово `goto`, которое никак не используется, из-за чего поле пришлось назвать `go_to`.) Для перехода на другую «метку» нужно присвоить новое значение полю класса `go_to` и перейти на новую итерацию при помощи `continue`;

Конструктор класса устанавливает `go_to` в 0, т.е. на самый первый базовый блок.

Базовые блоки состоят, как правило, из команд сопоставления с образцом, после которых может следовать построение результатного выражения или переход на другой базовый блок. Исключение — базовый блок, соответствующий невозможности отождествления, он генерирует исключение `RecognitionImpossible`.

Особенность базового блока в том, что все команды отождествления в нём при неудаче делают переход на один и тот же другой базовый блок. Это либо базовый блок, с которого начинается отождествление со следующим предложением, либо базовый блок, который делает попытку удлинить открытую e -переменную (попытка удлинения тоже может потерпеть неудачу, которая вызовет переход на какой-то ещё базовый блок).

Поэтому в начале базового блока переменной `go_to` присваивается номер блока — перехватчика неудачи, а команды отождествления при неудаче просто выполняют `continue`;

Рассмотрим в качестве примера отождествление с литерой '@' слева. Пусть литера '@' встретилась в образце `1 '@' e.X Hi`, макроцифра `1` и символ-слово `Hi` уже отождествлены и их адреса хранятся в `e[0]` и `e[1]`. Текущий срез — `s[0]`. Следующий свободный элемент — `e[2]`, в него будет присвоен индекс литеры '@'. Код будет выглядеть примерно так:

```
e[2] = e[0] + 1;
if (e[2] >= e[1]) continue;
if (! (s[0].at(e[2]) instanceof Character)) continue;
if ('@' != (char) s[0].at(e[2])) continue;
```

В принципе, все три условия можно объединить связкой `&&`, либо вообще их код вынести во вспомогательную функцию рантайма (что эффективнее — покажет эксперимент).

Первая проверка говорит о том, что литера '@' в выражении находится слева от слова `Hi`. Вторая — что это символ-литера. Третья — что у неё конкретное значение. В принципе, можно обе проверки записать одним условием:

```
if (! s[0].at(e[2]).equals('@')) continue;
```

но это потребует лишнюю упаковку, хоть и выглядит нагляднее.

Сопоставление с образцом делается по диссертации Романенко, но с учётом особенностей массивного представления.

В сгенерированном коде все индексы массивов константны, поэтому массивы `s` и `e` можно заменить переменными `s0`, `s1`, ... и `e0`, `e1`, ... , массивы были использованы здесь для наглядности изложения.

Сопоставление с открытой переменной требует нескольких базовых блоков. Пусть у нас сопоставляется образец `Left e.X <P> Right`, где `<P>` — некий подобразец. Пусть `Left` и `Right` находятся в `e[1]` и `e[2]` соответственно. Открытая переменная будет храниться в индексах `e[3]` и `e[4]`. Тогда код может выглядеть примерно так:

```
case <N>:
    go_to = <M>;
    <отождествление Left и Right>
    e[3] = e[1] + 1;
    e[4] = e[3] - 1; /* инициализируем как пустую, указатели внахлёт */
    go_to = <N+1>;
    continue;

case <N+1>:
    go_to = <N+2>;
    <отождествление P в диапазоне (e[4], e[2])>
    ...

case <N+2>:
    go_to = <M>;
    e[4] += 1;
    if (e[4] >= e[2]) continue;
    go_to = <N+1>;
    continue
```

Компиляция условий

Вспомним, как компилируется условие:

```
F {  
  ...  
  
  [m1 : P1, m2 : P2, ... ]  
    , <G1 u.11, u.12, ... > :: [v.11, v.12, ... ]  
  ...  
    , <Gk u.k1, u.k2, ... > :: [v.k1, v.k2, ... ]  
    ? [w.1, w.2, ... ] :: [Q1, Q2, ... ]  
    , <Gk' u.k'1, u.k'2, ... > :: [v.k'1, v.k'2, ... ]  
  ...  
  = u.r1, u.r2, ... ;  
  
  ...  
}
```

Для условия порождается вспомогательная функция F' , которая помнит состояние отождествления с образцом функции F . В случае неудачи сопоставления с образцом условия состояние используется для возобновления функции F (перехода к следующему предложению или удлинению е-переменной), в случае успеха — построения результатного выражения.

```
<G1 u.11, u.12, ... > :: [v.11, v.12, ... ],  
...  
<Gk u.k1, u.k2, ... > :: [v.k1, v.k2, ... ],  
<F' {состояние}, w.1, w.2, ... > :: [x.1, x.2, ... ],  
...  
{стек}
```

В роли функции F' будет выступать вызов функции F с `go_to != 0`. В роли состояния — поля текущего экземпляра вызова `e`, `s` и `args`. У этого вызова необходимо модифицировать массив `args`, чтобы он принимал значения левой части условия w_i , но при этом нельзя терять и исходные образцы.

Компиляция будет выглядеть примерно так:

```
case <N>:  
  args = new Object[] { args, null, ... , null };  
  runtime.push(this);  
  <создаём прочие элементы льеzona>  
  go_to = <N+1>;  
  return;  
  
case <N+1>:  
  go_to = <N+2>;  
  <сопоставляем аргументы условия с образцами>  
  ...
```

```

case <N+2>:
    args = (Object[]) args[0];
    go_to = <M>;
continue;

```

В `args[0]` сохраняем предыдущее значение `args`, оно потребуется при переходе на следующие предложения (да и небесполезно при отладке). По метке `<M>` находится код, который должен выполняться при неудаче условия — следующее предложение либо код удлинения открытой переменной. Для него нужно восстановить старое значение `args`. Под меткой `<N+1>` восстанавливать его нельзя, т.к. нужно отождествить аргументы условия, лежащие в новом `args`.

Компиляция функций-выражений

В каноническом лезонном представлении аргументами функций могут быть только переменные, либо объектные выражения. Аргументы функций, являющиеся выражениями с переменными, неявно преобразуются в вызовы специальных функций, которые принимают значения переменных в нём и строят соответствующее выражение.

Заметим, что выходной формат этих функций — либо `e`, либо `t`, причём во втором случае функция-выражение порождает скобочный терм. Действительно, если аргументом функции является символ, то он может быть записан либо литералом объектного символа, либо именем переменной. Если аргументом является терм, то он может быть либо символом, тогда см. выше, либо `t`-переменной, либо скобочным термом. А скобочный терм представляется просто как срез.

Для функций-выражений разумно предусмотреть специальный базовый класс

```

abstract class ExprFunction extends Function {
    public final String expr;

    public ExprFunction(String expr, int nargs) {
        this.expr = expr;
        args = new Object[nargs];
        returns = new ReturnValue[1];
        go_to = 0;
    }

    @Override
    void eval(Runtime runtime) {
        ReturnValue.assign(returns[0], makeExpr());
    }

    abstract Slice makeExpr();
}

```

Конструктор принимает имя функции (для выдачи в отладочном дампе) и число аргументов. Функция-выражение строится как безымянная функция

```

func = new ExprFunction("A (e.0 t.1) t.1 () e.2", 3) {
  @Override
  Slice makeExpr() {
    return r5jrt.Slice.concat(
      r5jrt.Slice.fromTerm('A'),
      r5jrt.Slice.fromTerm(
        r5jrt.Slice.concat(
          (r5jrt.Slice) args[0],
          r5jrt.Slice.fromTerm(args[1])
        )
      ),
      r5jrt.Slice.fromTerm(args[1]),
      r5jrt.Slice.fromTerm(r5jrt.Slice.EMPTY),
      (r5jrt.Slice) args[2]
    );
  }
};

```

Построение лезонной формы и повышение местности

Программа компилируется в несколько этапов:

0. Получение абстрактного синтаксического дерева — он выполняется библиотекой `refal-5-framework`, комментировать тут нечего.
1. Построение тривиальной лезонной формы.
2. Вычисление форматов функций.
3. Повышение местности.
4. Оптимизация условий.
5. Построение канонической лезонной формы.
6. Генерация кода.

Построение тривиальной лезонной формы

Тривиальная лезонная форма — форма, в которой все звенья (кроме вызовов встроенных функций) имеют вид

`<F <одно выражение>> :: e.i`

Форматы встроенных функций компилятор знает, поэтому может сразу же устанавливать правильные форматы для правых частей. Например, выражение

... `<F <First <аргумент>>> ...`

преобразуется в

```

...
<First <аргумент>> :: [e.i, e.<i+1>],
<F (e.i) e.<i+1>> :: e.<i+2>,
...

```

Несколько встроенных функций возвращают свой аргумент (или его часть) как результат (или его часть): `Print`, `Put`, `Lenw` и `Type`. Компилятор их может преобразовывать в эквивалентный код, где повторяющаяся часть вынесена отдельно:

```
<Print e.X>    → <Prout e.X> e.X
<Put s.N e.X> → <Putout s.N e.X> e.X
<Type e.X>     → <Type' e.X> e.X
<Lenw e.X>     → <Lenw' e.X> e.X
```

Здесь подразумевается, что функции `Type'` и `Lenw'` возвращают, соответственно, два символа — тип и подтип и один символ — длину.

Либо можно рассматривать эти функции как некие «полиморфные» форматы. Подобные «полиморфные» форматы будут актуальны и для функций `Chr`, `Lower`, `Ord` и `Upper`. Выходной формат вызовов этих функций будет совпадать с форматом аргумента с учётом соответствующего перекодирования символов, например,

```
<Upper e._ (s._ t._) 'Abc' (e._ 1 X)> == e._ (s._ t._) 'ABC' (e._ 1 X)
```

(Использована нотация безымянных переменных из Рефала-5λ.)

Вызовы «полиморфных» функций будут компилироваться в вызовы с кортежами параметров и возвращаемых значений переменной длины (при этом их длины будут равны). Для примера выше лезон будет выглядеть как

```
<Upper e.1, s.2, t.3, e.4> :: [e.5, s.6, t.7, e.8]
```

Однако, таким образом можно скомпилировать только вызовы `Chr`, `Lower`, `Ord` и `Upper`, вызовы `Print`, `Put`, `Lenw` и `Type` так преобразовать не получится. Но их можно преобразовать по схеме, описанной ранее (`Print` → `Prout` и т.д.). Так что оба способа анализа некоторых встроенных функций друг друга дополняют.

Побочным эффектом «полиморфных» форматов может оказаться вычисление функций с константными аргументами на стадии компиляции, например, `<Ord '0'> → 48` (лезон `<Ord> :: []` можно заменить на инкремент счётчика шагов рефал-машины).

Вычисление форматов функций

Повышение местности основывается на алгоритме, предложенном в соответствующей статье Романенко 1990 года. Нам потребуются алгоритмы обобщённого сопоставления с образцом и обобщения двух параметризованных выражений, оба алгоритма описаны в соответствующих разделах.

На алгоритм обобщения мы будем ссылаться как на функцию $GEN(E_1, \dots, E_n)$, принимающую произвольное число параметров. Результатом функции является жёсткое выражение.

Жёсткое выражение — выражение, которое описывается следующим синтаксисом:

$$\begin{aligned} He &\rightarrow \emptyset | He' \\ He' &\rightarrow e | \varepsilon | HtHe' | He'Ht \\ Ht &\rightarrow X | s | t | (He') \end{aligned}$$

Здесь X — произвольный литерал символа, e, s, t — переменные без индексов, Ht — жёсткий терм, He' — вспомогательный нетерминал.

Как видим, жёстким выражением может быть и пустое множество.

Вообще, пустое множество также является допустимым множеством объектных выражений и для него справедливы аксиомы:

- $E\emptyset = \emptyset E = \emptyset$ — конкатенация с пустым множеством является пустым множеством,
- $(\emptyset) = \emptyset$ — скобочный терм с пустым множеством внутри сам пустое множество,
- $GEN(\overline{He_i}, \emptyset, \overline{He_j}) = GEN(\overline{He_i}, \overline{He_j})$ — пустое множество — нейтральный элемент относительно операции обобщения,
- $GEN(\emptyset) = \emptyset$.

Здесь запись $\overline{\text{Формула}_i}$ означает повторение формулы под чертой некоторое количество раз: $\text{Формула}_1, \dots, \text{Формула}_n$. Это обозначение будет использоваться и в дальнейшем.

Форматы функций вычисляются при помощи абстрактной интерпретации.

Входная точка программы конструируется искусственно как новая функция с уникальным именем (имя её не важно), включающая все предложения всех функций компилируемого файла, за исключением функций, созданных при рассаживании блоков.

Позже увидим, что если в файле отсутствуют вызовы метафункций (Ev-met, Mu, Residue и Up), то во входную точку достаточно включить все предложения всех функций, помеченных как §ENTRY.

Всем внешним функциям назначается формат $e = e$.

Входной точке назначается формат $e = e$ (на самом деле, её выходной формат не важен), всем остальным функциям — $\emptyset = \emptyset$.

Затем до неподвижной точки выполняются следующие шаги:

- Выполняется проход по всем функциям в программе, включая и фиктивную входную точку.
 - Для каждой функции F выполняется проход по всем предложениям.
 - * Для каждого предложения выполняется сопоставление входного формата с образцом предложения. Значениями переменных становятся параметризованные выражения.
 - * Для каждого звена лезона выполняются следующие действия:
 - Если значением хотя бы одной переменной является $\emptyset$, то остаток предложения заменяется на $= \emptyset$.
 - Если это вызов функции $\langle G \dots \rangle :: e_X$, вместо переменных подставляются их значения, лезонная переменная e_X получает значение выходного формата G , переменные из формата получают свежие индексы.
 - Если это условие $? E : P$, то в выражения E и P подставляются значения переменных и выполняется сопоставление с образцом, которое назначает переменным новые значения. Если сопоставление невозможно, то часть предложения, начиная с этого условия заменяется на $\$fail$.
 - * Значения переменных подставляются в возвращаемое значение предложения $= R$ — получаем выходной формат выражения.
 - Вычисляется выходной формат функции F как обобщение выходных форматов каждого предложения.
- Вычисляется входной формат каждой из функций как обобщение всех фактических аргументов всех вызовов этой функции.

Форматы встроенных функций в лезонах могут обрабатываться особым образом, как описано в предыдущих разделах.

Алгоритмы обобщённого сопоставления с образцом

На самом деле, это не один алгоритм, а три:

1. Точное сопоставление формата функции с образцом — может давать 0 и более решений, используется при повышении местности.
2. Неточное сопоставление параметризованного выражения с образцом — может давать 0 или 1 решение, используется при вычислении форматов (во всех случаях в предыдущем разделе) и оптимизации условий.
3. Сопоставление параметризованного выражения с форматом — всегда имеет единственное решение, т.к. используется только там, где такое решение существует.

Далее будем использовать следующие обозначения:

- X, Y — литералы символов, если в формуле одновременно упоминаются и X , и Y , то подразумевается, что $X \neq Y$,
- e, t, s — безымянные переменные, могут быть только в жёстких выражениях,
- e_x, t_x, s_x — переменные,
- E — произвольное параметризованное выражение, T — произвольный параметризованный терм,
- P — образцовое выражение, Pt — образцовый терм,
- He — жёсткое выражение, Ht — жёсткий терм.

Синтаксической разницы между E и P , а также, между T и Pt нет, разные обозначения используются, чтобы подчеркнуть разницу в понятиях.

Точное сопоставление формата функции с образцом $He : P$

Входными данными для алгоритма служат входной формат функции He и предложение $PTail$, где P — образец до повышения местности, $Tail$ — цепочка из лезнов и условий, заканчивающаяся возвратом значения.

Выходными данными являются несколько предложений (ноль и более) вида $\overline{[m_i : P_i]} Tail'$, где m_i — это тип аргумента e, t или s , P_i — образец для соответствующего аргумента, а $Tail'$ — это преобразованное окончание предложения.

В процессе мы будем рассматривать уравнения вида

$$\overline{[He_i : P_i]} Tail$$

Сопоставление $He_i : P_i$ мы будем называть *клэшем*.

В процессе преобразований уравнение может порождать 0, 1 или 2 новых уравнения. Если новых уравнений два, то их порядок важен.

Сопоставление начинается с уравнения $[He : P] Tail$ и продолжается до тех пор, пока He_i не обратятся в переменные m_i .

Запись $[Cl', He : P, Cl''] Tail$ означает уравнение с выделенным клэшем, Cl' и Cl'' означают клэши до и после выделенного.

Запись v_* означает переменную со свежим индексом — индексом, который в предложении ранее не встречался.

Запись $[Cl] Tail/\sigma$ означает выполнение подстановки:

$$\overline{[He_i : P_i]} Tail/\sigma = \overline{[He_i : P_i/\sigma]} (Tail/\sigma)$$

Ну, собственно, правила. Они общеизвестны (см. статьи Турчина 1972 и 1974 годов).
Клэши вида «выражение : выражение»:

$$\begin{aligned}
[Cl', Ht He : Pt P, Cl''] Tail &\rightarrow [Cl', Ht : Pt, He : P, Cl''] Tail \\
[Cl', He Ht : P Pt, Cl''] Tail &\rightarrow [Cl', He : P, Ht : Pt, Cl''] Tail \\
[Cl', Ht He : e_X P, Cl''] Tail &\rightarrow \begin{cases} [Cl', Ht He : e_X P, Cl''] Tail / \{e_X \rightarrow \epsilon\} \\ [Cl', Ht He : e_X P, Cl''] Tail / \{e_X \rightarrow t_{\star_1} e_{\star_2}\} \end{cases} \\
[Cl', He Ht : P e_X, Cl''] Tail &\rightarrow \begin{cases} [Cl', He Ht : P e_X, Cl''] Tail / \{e_X \rightarrow e_{\star_1} t_{\star_2}\} \\ [Cl', He Ht : P e_X, Cl''] Tail / \{e_X \rightarrow \epsilon\} \end{cases} \\
[Cl', \epsilon : \epsilon, Cl''] Tail &\rightarrow [Cl', Cl''] Tail \\
[Cl', \epsilon : e_X P, Cl''] Tail &\rightarrow [Cl', \epsilon : e_X P, Cl''] Tail / \{e_X \rightarrow \epsilon\} \\
[Cl', \epsilon : Pt P, Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', \epsilon : P Pt, Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', Ht He : \epsilon, Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', He Ht : \epsilon, Cl''] Tail &\rightarrow \text{решений нет}
\end{aligned}$$

Клэши вида «терм : терм»:

$$\begin{aligned}
[Cl', (He) : (P), Cl''] Tail &\rightarrow [Cl', He : P, Cl''] Tail \\
[Cl', (He) : t_X, Cl''] Tail &\rightarrow [Cl', (He) : t_X, Cl''] Tail / \{t_X \rightarrow (e_{\star})\} \\
[Cl', (He) : s_X, Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', (He) : X, Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', s : (P), Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', s : t_X, Cl''] Tail &\rightarrow [Cl', s : t_X, Cl''] Tail / \{t_X \rightarrow s_{\star}\} \\
[Cl', X : (P), Cl''] Tail &\rightarrow \text{решений нет} \\
[Cl', X : t_X, Cl''] Tail &\rightarrow [Cl', X : t_X, Cl''] Tail / \{t_X \rightarrow X\} \\
[Cl', X : s_X, Cl''] Tail &\rightarrow [Cl', X : s_X, Cl''] Tail / \{s_X \rightarrow X\} \\
[Cl', X : X, Cl''] Tail &\rightarrow [Cl', Cl''] Tail \\
[Cl', X : Y, Cl''] Tail &\rightarrow \text{решений нет}
\end{aligned}$$

Неточное сопоставление параметризованного выражения с образцом $E : P$

Входными данными алгоритма являются параметризованное выражение E и образец P .

Выходными данными алгоритма является набор клэшей $[E_i : P_i]$ либо $\$fail$.

Сопоставление начинается с единственного клэша $[E : P]$, затем применяются следующие правила до тех пор, пока это возможно.

Клэши вида «выражение : выражение»:

$$\begin{aligned}
[Cl', E : E, Cl''] &\rightarrow [Cl', Cl''] \\
[Cl', TE : PtP, Cl''] &\rightarrow [Cl', T : Pt, E : P, Cl''] \\
[Cl', ET : PPt, Cl''] &\rightarrow [Cl', E : P, T : Pt, Cl''] \\
[Cl', \epsilon : e_X P, Cl''] &\rightarrow [Cl', \epsilon : e_X, \epsilon : P, Cl''] \\
[Cl', e_X E : \epsilon, Cl''] &\rightarrow [Cl', e_X : \epsilon, E : \epsilon, Cl''] \\
[Cl', \epsilon : PtP, Cl''] &\rightarrow \$fail \\
[Cl', TE : \epsilon, Cl''] &\rightarrow \$fail
\end{aligned}$$

Клэши вида «терм : терм»:

$$\begin{aligned}
[Cl', T : T, Cl''] &\rightarrow [Cl', Cl''] \\
[Cl', (E) : (P), Cl''] &\rightarrow [Cl', E : P, Cl''] \\
[Cl', (E) : s_X, Cl''] &\rightarrow \$fail \\
[Cl', s_X : (P), Cl''] &\rightarrow \$fail \\
[Cl', (E) : X, Cl''] &\rightarrow \$fail \\
[Cl', X : (P), Cl''] &\rightarrow \$fail
\end{aligned}$$

Первые строчки в обеих группах правил соответствуют тривиальным клэшам, где и слева, и справа записано одинаковое выражение (одинаковый терм). Такой клэш просто стирается.

Алгоритму повышения местности и оптимизации выражений достаточно получить набор оставшихся клэшей. Алгоритму вывода форматов также нужны значения переменных.

Новые значения переменных определяются следующим образом:

- Для клэша вида $v_i : v_j$ строится подстановка вида $v_i \rightarrow v_j$ или $v_j \rightarrow v_i$ в зависимости от того, какая из переменных «шире». В левой части подстановки не может быть более узкой переменной (т.е. $s_X \rightarrow e_Y$ недопустима, должно быть $e_Y \rightarrow s_X$).
- Для клэшей вида $v_i : E$ или $P : v_j$ строится подстановка вида $v_i \rightarrow E$ или $v_j \rightarrow P$ соответственно.
- Для всех переменных в клэшах вида $E : P$ строятся тривиальные подстановки $v_i \rightarrow v_i$.

Если для переменной получено более одной подстановки — она является повторной. В этом случае все подстановки для неё стираются. Выглядит парадоксально (отбрасываем найденную информацию), но это упростит реализацию следующего этапа — повышения местности.

Сопоставление параметризованного выражения с форматом $E : He$

Сопоставление выполняется только в случае, когда E действительно соответствует формату He .

Входные данные: E и He .

Выходные данные: кортеж выражений $[E_i : m_i]$.

Преобразование начинается с кортежа $[E : He]$ и выполняется по следующим правилам до тех пор, пока они остаются применимыми:

$$\begin{aligned}
[Cl', T E : Ht He, Cl''] &\rightarrow [Cl', T : Ht, E : He, Cl''] \\
[Cl', ET : He Ht, Cl''] &\rightarrow [Cl', E : He, T : Ht, Cl''] \\
[Cl', \epsilon : \epsilon, Cl''] &\rightarrow [Cl', Cl''] \\
[Cl', (E) : (He), Cl''] &\rightarrow [Cl', E : He, Cl''] \\
[Cl', X : X, Cl''] &\rightarrow [Cl', Cl'']
\end{aligned}$$

Алгоритм построения обобщения

Сложностью жёсткого выражения назовём следующую характеристику:

$$C = 1 + n_t + 2n_s + 3n_X + 3n_{()} - n_e$$

где n_t — число t-переменных, n_s — число s-переменных, n_X — число литералов символов, $n_{()}$ — число пар скобок, n_e — число e-переменных.

Можно показать, что любая из следующих элементарных подстановок

$$\begin{aligned}
e &\rightarrow te \quad | \quad et \quad | \quad \epsilon \\
t &\rightarrow (e) \quad | \quad s \\
s &\rightarrow X
\end{aligned}$$

увеличивает сложность жёсткого выражения на 1.

Функция $GEN(\overline{E_i})$ вычисляет жёсткое выражение, которое можно преобразовать подстановками в любой из аргументов функции.

При этом будем полагать, что $GEN(\emptyset) = GEN() = \emptyset$.

Введём функцию $GEN_T(\overline{T_i})$, которая вычисляет обобщение термов.

Алгоритм вычисления $GEN(\overline{E_i})$:

- Если все аргументы можно представить и в виде $\overline{E_i} = \overline{T'_i E'_i}$, и в виде $\overline{E_i} = \overline{E''_i T''_i}$, то

$$GEN(\overline{E_i}) = \begin{cases} T' GEN(\overline{E'_i}), & \text{если } C(T') \geq C(T'') \\ GEN(\overline{E''_i}) T'', & \text{в противном случае} \end{cases}$$

где

$$T' = GEN_T(\overline{T'_i}) \quad \text{и} \quad T'' = GEN_T(\overline{T''_i})$$

- Если все аргументы можно представить в виде $\overline{E_i} = \overline{T'_i E'_i}$, то

$$GEN(\overline{E_i}) = GEN_T(\overline{T'_i}) GEN(\overline{E'_i})$$

- Если все аргументы можно представить в виде $\overline{E_i} = \overline{E''_i T''_i}$, то

$$GEN(\overline{E_i}) = GEN(\overline{E''_i}) GEN_T(\overline{T''_i})$$

- $GEN(\overline{\epsilon}) = \epsilon$, т.е. все аргументы — пустые,
- $GEN(\overline{E_i}) = e$ в остальных случаях.

Алгоритм вычисления $GEN_T(\overline{T_i})$:

- $GEN_T(\overline{(E_i)}) = (GEN(\overline{E_i}))$ — все аргументы — скобочные термы,
- $GEN_T(\overline{X}) = X$ — все аргументы — одинаковые символы,
- $GEN_T(\overline{S_i}) = s$ — все аргументы — или символы, или s-переменные,
- $GEN_T(\overline{T_i}) = t$ в остальных случаях.

Повышение местности и оптимизация условий

Введём функцию $Indices(He)$, которая назначает свежие индексы переменным из жёсткого выражения:

$$\begin{aligned} Indices((He') He'') &= (Indices(He')) Indices(He'') \\ Indices(v He) &= v_* Indices(He), \quad \text{где } v \in \{s, t, e\} \\ Indices(X He) &= X Indices(He) \\ Indices(\epsilon) &= \epsilon \end{aligned}$$

Введём функцию $Vars(E)$, которая строит кортеж из всех переменных выражения:

$$\begin{aligned} Vars((E') E'') &= Vars(E') + Vars(E'') \\ Vars(v_X E) &= [v_X] + Vars(E), \quad \text{где } v \in \{s, t, e\} \\ Vars(X E) &= Vars(E) \\ Vars(\epsilon) &= [] \end{aligned}$$

Будем обозначать $In[F]$ и $Out[F]$ — входной и выходной форматы функции F .

Алгоритм следующий:

- Для каждого предложения $PTail$ функции F разрешаем клэш $[In[F] : P] Tail$ — получаем несколько предложений вида $[\overline{m_i} : \overline{P_i}] Tail$.
- Затем перебираем все звенья в $Tail$:

– Для звена $\langle G arg \rangle :: e_X$, если $Out[G] = \emptyset$, строим связку

$$\langle G arg' \rangle :: []$$

где $arg' = arg : In[G]$, остаток предложения отбрасываем.

– Для звена $\langle G arg \rangle :: e_X$ строим новую связку

$$\langle G arg' \rangle :: [\overline{v_j}]$$

где

$$\begin{aligned} arg' &= arg : In[G], \\ res &= Indices(Out[G]), \\ [\overline{v_j}] &= Vars(res) \end{aligned}$$

Подставляем $e_X \rightarrow res$ в оставшуюся часть предложения.

– Для звена $? E : P$ выполняем неточное сопоставление с образцом:

- * Если $E : P = \$fail$, заменяем звено на $fail$ и отбрасываем остаток предложения.
- * Если $E : P = [\overline{E_i} : \overline{P_i}]$, то перебираем клэши:

- Клэши вида $E_i : v_i$, где v_i — неповторная переменная, удаляем, подстановку $v_i \rightarrow E_i$ применяем к остатку предложения.
- Для клэшей вида $v_i : P_i$, где v_i — неповторная переменная, только применяем подстановку $v_i \rightarrow P_i$ к остатку предложения.
- Если условие нетривиальное (т.е. кортеж остался непустым), строим звено-условие ? $[E_i : P_i]$.
- Результатное выражение $= E^R$, после подстановок переменных, должно соответствовать формату $Out[F]$. Сопоставляем E^R с форматом

$$[\overline{E_i^R : m_i}] = E^R : Out[F]$$

и строим результатное выражение $= [\overline{E_i^R : m_i}]$.

Если образец условия не содержит повторных переменных и соответствует по формату левой части, то условие, фактически, превращается в лезон. Оптимизация условий позволяет использовать их во входном языке в роли лезонов без накладных расходов на организацию условий. В противоположность, в Рефале Плюс условия и перестройки синтаксически и семантически различаются.

Генерация кода

Форматные и бесформатные функции

Тривиальным будем называть формат $e = e$.

Будем говорить, что функция **бесформатная**, если её формат — тривиальный. Соответственно, функция с нетривиальным форматом — **форматная**.

По умолчанию, каждая форматная функция программы компилируется в пару функций:

- бесформатную (будем обозначать её $\langle \text{функ} \rangle$) — её реализация в точности соответствует реализации в исходном тексте с точностью до повышения местности вызовов в ней,
- форматную (будем обозначать её $\langle \text{функ} \rangle.\text{fmt}$) — специализированную по обобщению форматов аргументов всех прямых вызовов этой функции.

Если вычисленный формат функции $e = e$, то генерируется только бесформатное её воплощение.

Заметим, что форматное и бесформатное воплощения форматной функции в общем случае не эквивалентны. Форматное воплощение *специализировано*, т.е. её реализация учитывает то, что она всегда вызывается с каким-то подмножеством аргументов.

Прямые вызовы форматных функций компилируются в вызовы их форматных воплощений. Бесформатное воплощение может вызываться в двух случаях:

- если это entry-функция и она вызывается извне,
- и если функция вызывается косвенно, через метафункцию (Ev-met, Mu, Residue или Up).

Если в файле ни одна метафункция не вызывается, то бесформатные воплощения для форматных функций локальной области видимости можно не генерировать.

В сгенерированном коде форматные функции генерируются как вложенный статический класс с именем `fmt`:

```

class < функ > {
    ...

    static class fmt {
        ...
    }
}

```

Поскольку при декорировании в конец имён классов для функций добавляется `_`, имя `fmt` не приведёт ни к каким конфликтам. Для функций локальной области видимости получается двойная вложенность:

```

class Entry {
    /* бесформатное воплощение функции Entry */
    ...

    static class fmt {
        /* форматное воплощение функции Entry */
        ...
    }

    static class Local {
        /* бесформатное воплощение функции Local */
        ...

        static class fmt {
            /* форматное воплощение функции Local */
            ...
        }
    }
}

```

Встроенные функции поддерживают три воплощения:

- бесформатное воплощение для косвенного вызова из метафункций — `< функ >`,
- «полуформатное» воплощение с тривиальным входным и нетривиальным выходным форматом — `< функ >.out`,
- форматное воплощение, если фактический аргумент удовлетворяет её формату — `< функ >.fmt`.

Компилятор форматы функций знает и при построении лъезонной формы сразу учитывает выходные форматы, поэтому бесформатный вызов может быть только косвенным.

Для примера, `<Mu First e.X>` вызовет `First`, `<First <Add 1 1> e.X>` — `First.out` (т.к. выходной формат арифметической функции `e`), `<First s.P e.X>` — `First.fmt`.

Компиляция метафункций

Метафункции уместно рассматривать как функции локальной области видимости, неявно добавляемые компилятором в каждую единицу трансляции (Рефал-5 и Рефал-5λ именно так и поступают). В реализации для Java разумно поступать также.

Метафункция должна уметь находить по имени соответствующую функцию и её вызывать. При этом, в первую очередь она должна просматривать локальную область видимости (все функции в текущем файле, включая метафункции), а затем — глобальную (все entry-функции программы, включая встроены функции).

Поиск в глобальной области видимости можно осуществить средствами рефлексии языка Java:

- проверить, что имя является допустимым именем функции Рефала,
- декорировать имя (добавив "r5jrt." в начало),
- найти класс по имени и создать.

Проверка и декорирование имени могут выполняться одновременно.

Все локальные функции при компиляции попадают в класс первой entry-функции файла, в том числе и метафункции. Поэтому локальную функцию тоже технически можно найти средствами рефлексии — узнать, в каком классе находится класс метафункции (как полное имя класса минус суффикс из имени метафункции и, возможно, .fmt или .out) и использовать имя этого класса как префикс при поиске.

Рефлексия — дорогой механизм, поэтому поиск можно оптимизировать, кэшируя найденные классы (вернее, их конструкторы). Причём кэш достаточно иметь один на все 4 метафункции и разместить его можно в любой из них (для определённости — в Mu пусть будет).

Если в файле используется одна метафункция, то потенциально могут быть вызваны и все остальные. Например, <Mu (<Card>) ...> может вызвать, например, Ev-met, если соответствующее слово прочитается со стандартного ввода.

Поэтому, если компилятор для оптимизации не порождает метафункции, если они не вызываются, то должен генерировать или все, или ни одной. И, кстати, поэтому кэш можно хранить в любой из них.

Из соображений оптимизации кэш можно заполнять заранее именами всех встроенных и локальных функций.

Прочее

\$fail компилируется в безусловный переход:

```
case <N>:
  go_to = <M>;
  continue;
```

Оптимизация хранения

Внутреннее представление среза как `Object[]` может быть накладным: для доступа к значениям примитивных типов требуется упаковка и распаковка, а также проверка типа значения.

Можно попробовать оптимизировать представление, храня литеры и макроцифры в целочисленном массиве, а строки и скобочные термы — в ссылочном:

```

class Slice {
    long[] ival;
    Object[] oval;
    int left, right;

    static final SYMBOL = -(1L << 32) - 1;
    static final SLICE = -(1L << 32) - 2;

    ...

    bool isCharAt(int pos, int code) {
        return ival[pos + left] == code;
    }

    bool isIntAt(int pos, long value) {
        return ival[pos + left] == -(value - 1);
    }

    bool isStringAt(int pos, String value) {
        return ival[pos + left] == SYMBOL
            && (oval[pos + left] == value || value.equals(oval[pos + left]));
    }

    bool isBracketsAt(int pos) {
        return ival[pos + left] == SLICE;
    }

    Slice bracketsAt(int pos) {
        assert isBracketsAt(pos);
        return (Slice) oval[pos + left];
    }

    ...
}

```

- Литеры представлены как неотрицательные значения массива `ival`.
- Макроцифры представлены как отрицательные значения: $0 \dots 2^{32} - 1 \rightarrow -1 \dots -2^{32}$.
- Значение $-2^{32} - 1$ используется для любых других символов (включая строки).
- Значение $-2^{32} - 2$ используется для скобочных термов.

Сопоставление с образцом выполняется при помощи методов `is...At(...)`. Будет ли это представление эффективнее и на сколько — нужны эксперименты.